

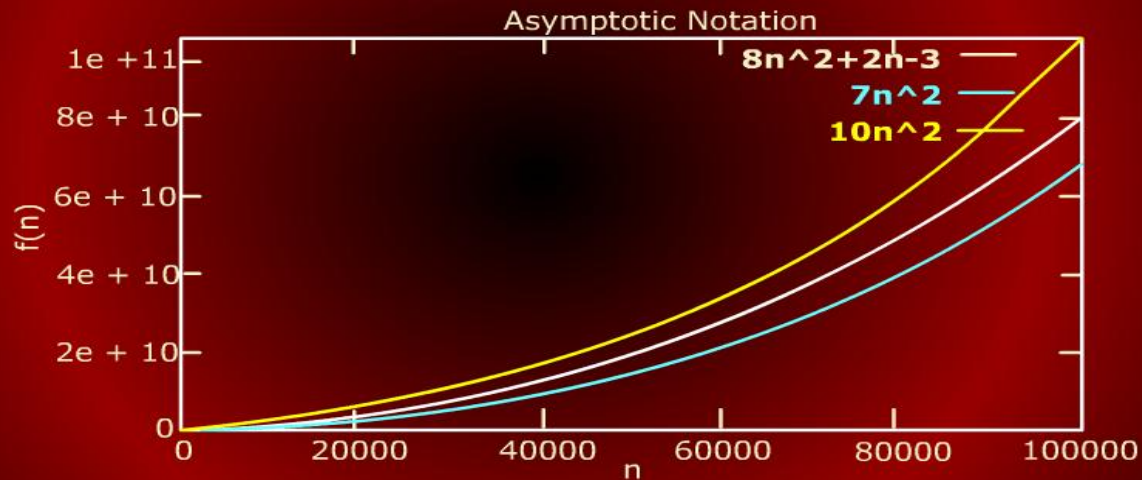
Algorithms

Lecture #07

Syed Hamed Raza

Asymptotic Notation - Example

Asymptotic Notation - Example



Asymptotic Notation - Example

Asymptotic Notation - Example

- We have established that $f(n) \in n^2$.
- Let's show why $f(n)$ is **not** in some other asymptotic class.
- First, let's show that $f(n) \notin \Theta(n)$.

Asymptotic Notation - Example

Asymptotic Notation - Example

show that $f(n) \notin \Theta(n)$.

- If this were true, we would have had to satisfy both the upper and lower bounds.
- The lower bound is satisfied because $f(n) = 8n^2 + 2n - 3$ does grow at least as fast asymptotically as n .

Asymptotic Notation - Example

Asymptotic Notation - Example

- The lower bound is satisfied because $f(n) = 8n^2 + 2n - 3$ does grow at least as fast asymptotically as n .
- But the upper bound is false. Upper bounds requires that there exist positive constants c_2 and n_0 such that $f(n) \leq c_2n$ for all $n \geq n_0$.

Asymptotic Notation - Example

Asymptotic Notation - Example

- Informally we know that $f(n) = 8n^2 + 2n - 3$ will eventually exceed c_2n no matter how large we make c_2 .
- To see this, suppose we assume that constants c_2 and n_0 did exist such that $8n^2 + 2n - 3 \leq c_2n$ for all $n \geq n_0$

Asymptotic Notation - Example

Asymptotic Notation - Example

- If we divide both sides by n , we have

$$\lim_{n \rightarrow \infty} \left(8n + 2 - \frac{3}{n} \right) \leq c_2.$$

- It is easy to see that in the limit, the left side tends to ∞ .
- So, no matter how large c_2 is, the statement is violated. Thus $f(n) \notin \Theta(n)$.

Asymptotic Notation - Example

Asymptotic Notation - Example

Let's show that $f(n) \notin \Theta(n^3)$.

- The idea would be to show that the lower bound $f(n) \geq c_1 n^3$ for all $n \geq n_0$ is **violated**.
- (c_1 and n_0 are positive constants).
- Informally we know this to be true because any cubic function will overtake a quadratic.

Asymptotic Notation - Example

Asymptotic Notation - Example

- If we divide both sides by n^3 :

$$\lim_{n \rightarrow \infty} \left(\frac{8}{n} + \frac{2}{n^2} - \frac{3}{n^3} \right) \geq c_1$$

- The left side tends to 0. The only way to satisfy this is to set $c_1 = 0$.
- But by hypothesis, c_1 is positive.
- This means that $f(n) \notin \Theta(n^3)$.

O-Notation...

O-notation and Ω notation

- The definition of Θ -notation relies on proving both a lower and upper asymptotic bound.
- Sometimes we only interested in proving one bound or the other.
- The **O-notation** is used to state only the asymptotic upper bounds.
- The **Ω -notation** allows us to state only the asymptotic lower bounds.

O-Notation (Big O)

O-notation (Big-O)

- $O(g(n)) = \{f(n) \mid \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n)$
 $\text{for all } n \geq n_0\}$

Notation

Ω -notation

- $\Omega(g(n)) = \{f(n) \mid \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n)$
 $\text{for all } n \geq n_0\}$

Notation

Θ , O , Ω -notation

- The three notations:

$$\Theta(g(n)) : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$O(g(n)) : 0 \leq f(n) \leq c g(n)$$

$$\Omega(g(n)) : 0 \leq c g(n) \leq f(n) \\ \text{for all } n \geq n_0$$

Limit Rule

Limit Rule

These definitions suggest an alternative way of showing the asymptotic behavior.

Limit rule for Θ -notation:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c,$$

for some constant $c > 0$ (strictly positive but not infinity) then

$$f(n) \in \Theta(g(n)).$$

Limit Rule

Limit Rule

Limit rule for O-notation:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c,$$

for some constant $c \geq 0$ (nonnegative but not infinite) then $f(n) \in O(g(n))$.

Limit Rule

Limit Rule

Limit rule for Ω -notation:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0,$$

(either a strictly positive constant or infinity) then $f(n) \in \Omega(g(n))$.

Asymptotic Intuition

Asymptotic Intuition

Here is a list of common asymptotic running times:

- $\Theta(1)$: Constant time; can't beat it!
- $\Theta(\log n)$: Inserting into a balanced binary tree; time to find an item in a sorted array of length n using binary search.
- $\Theta(n)$: About the fastest that an algorithm can run.

Asymptotic Intuition

Asymptotic Intuition

- $\Theta(1)$: Constant time; can't beat it!
- $\Theta(\log n)$: Inserting into a balanced binary tree; time to find an item in a sorted array of length n using binary search.
- $\Theta(n)$: About the fastest that an algorithm can run.
- $\Theta(n \log n)$: Best sorting algorithms.

Asymptotic Intuition

Asymptotic Intuition

- $\Theta(n^2)$, $\Theta(n^3)$: Polynomial time.
These running times are acceptable when the exponent of n is small or n is not too large, e.g., $n \leq 1000$.
- $\Theta(2^n)$, $\Theta(3^n)$: Exponential time.
Acceptable only if n is small, e.g., $n \leq 50$.
- $\Theta(n!)$, $\Theta(n^n)$: Acceptable only for really small n , e.g. $n \leq 20$.

Divide and Conquer

Divide and Conquer

The divide and conquer is a strategy employed to solve a large number of computational problems:

- **Divide:** the problem into a small number of pieces
- **Conquer:** solve each piece by applying divide and conquer to it recursively
- **Combine:** the pieces together into a global solution.

Merge Sort

Merge Sort

- Divide and conquer strategy is applicable in a huge number of computational problems.
- The first example of divide and conquer algorithm we will discuss is a simple and efficient sorting procedure called *Merge Sort*.

Merge Sort

Merge Sort

- We are given a sequence of n numbers A , which we will assume are stored in an array $A[1..n]$.
- The objective is to output a permutation of this sequence sorted in increasing order.
- This is normally done by permuting the elements within the array A .

Merge Sort

Merge Sort

Here is how the merge sort algorithm works:

- **Divide:** split A down the middle into two subsequences, each of size roughly $n/2$
- **Conquer:** sort each subsequence by calling merge sort recursively on each.

Merge Sort

Merge Sort

- **Divide:** split A down the middle into two subsequences, each of size roughly $n/2$
- **Conquer:** sort each subsequence by calling merge sort recursively on each.
- **Combine:** merge the two sorted subsequences into a single sorted list.

Merge Sort

Merge Sort

- The dividing process ends when we have split the subsequences down to a single item.
- A sequence of length one is trivially sorted.
- The key operation is the combine stage which merges together two sorted lists into a single sorted list.

Merge Sort

Merge Sort

- A sequence of length one is trivially sorted.
- The key operation is the combine stage which merges together two sorted lists into a single sorted list.
- Fortunately, the combining process is quite easy to implement.

Divide and Conquer Strategy

Divide and Conquer
Strategy

Merge Sort

Merge sort

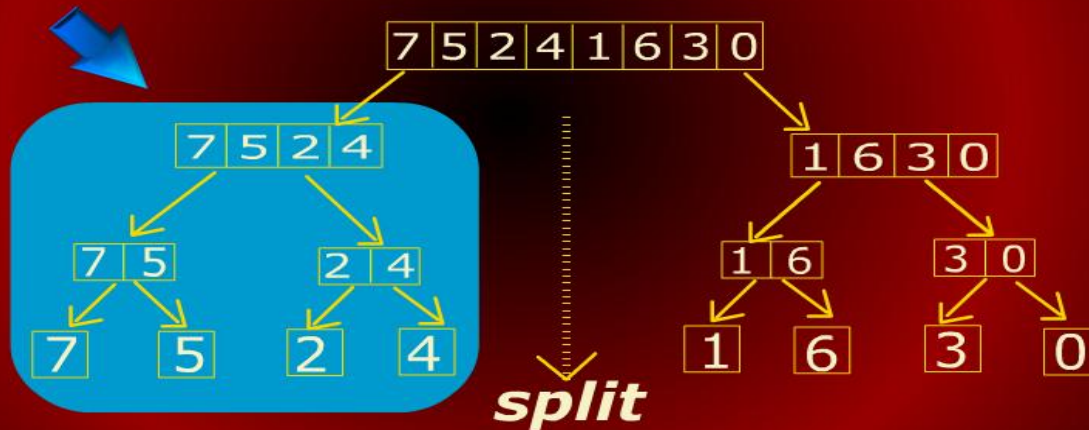
The divide phase: split the list top-down into smaller pieces



Merge Sort

Merge sort

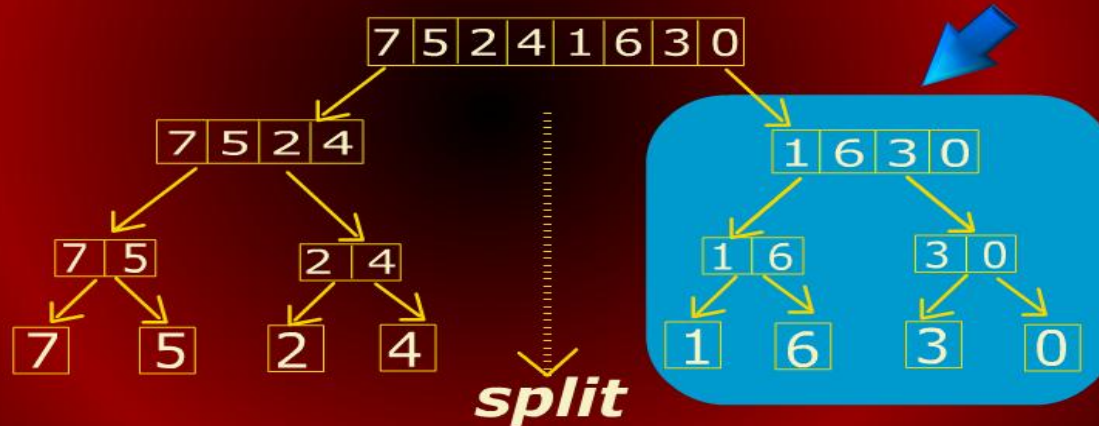
The divide phase: split the list top-down into smaller pieces



Merge Sort

Merge sort

The divide phase: split the list top-down into smaller pieces



Limit Rule

Merge Sort

Design and Analysis of Algorithm

Asymptotic Notation - Example

- To see this, suppose we assume that constants c_2 and n_0 did exist such that $8n^2 + 2n - 3 \leq c_2n$ for all $n \geq n_0$
- Since this is true for all sufficiently large n then it must be true in the limit as n tends to infinity.